APACHE HADOOP MAPREDUCE

Weather Data Analysis - Maximum Temperature per City

Name: Nidhi

USN: 4GW23CI035

Semester: 7th

Aim:

To write a MapReduce program in Python using the mrjob library that reads weather data stored in HDFS and finds the maximum temperature recorded in each city.

Problem Statement:

A weather department collects daily temperature readings from weather stations in different cities. Over many years this data becomes very large, so it is stored in the Hadoop Distributed File System (HDFS). The department wants to find the highest temperature recorded in each city, to study heat patterns and issue heat alerts.

In the Map phase, each record is split into fields and the program outputs the pair (city, temperature). Hadoop then groups all temperatures that belong to the same city. In the Reduce phase, the maximum temperature of each city is found.

Tools and Environment Used:

Operating System: Ubuntu 24.04 LTS (virtual machine)

Java: OpenJDK 17

Hadoop: Apache Hadoop 3.5.0

Language: Python 3.12

Library: mrjob (runs Python MapReduce jobs on Hadoop using Hadoop Streaming)

Input Data:

The input file weather.csv contains daily readings for three cities over three days. Each record has the date, city, temperature (in degrees Celsius) and humidity:

The file is stored in HDFS at: /user/nidhi/weather/weather.csv

Code Implementation:

File name: weather.py

date,city,temperature,humidity

2026-09-01,Mysore,29,65

2026-09-01,Bangalore,27,70

2026-09-01,Chennai,34,75

2026-09-02,Mysore,31,60

2026-09-02,Bangalore,26,72

2026-09-02,Chennai,36,78

2026-09-03,Mysore,28,68

2026-09-03,Bangalore,29,66

2026-09-03,Chennai,35,80

```
from mrjob.job import MRJob


class MaxTemperature(MRJob):

    def mapper(self, _, line):
        fields = line.split(',')
        if fields[0] == 'date':
            return
        city = fields[1]
        temperature = int(fields[2])
        yield city, temperature

    def reducer(self, city, temperatures):
        yield city, max(temperatures)


if __name__ == '__main__':
    MaxTemperature.run()
EOF
aiml@ubuntu:~$ cat ~/weather.py
```

mapper(): Splits each CSV line at the commas and skips the header row. The second field is the city and the third field is the temperature. It outputs the pair (city, temperature).

reducer(): Receives one city with the list of all its temperatures and outputs the maximum value using max().

run(): Starts the job. The option -r hadoop makes mrjob submit the job to the Hadoop cluster through Hadoop Streaming.

Execution Steps:

1. Start the Hadoop services and check that they are running:

>>start-dfs.sh
>>start-yarn.sh
>>jps

2. Upload the input file to HDFS:

>>hdfs dfs -mkdir -p /user/nidhi/weather
>>hdfs dfs -put -f ~/weather.csv /user/nidhi/weather/
>>hdfs dfs -ls /user/nidhi/weather

3. Activate the Python environment and run the job on Hadoop, saving the output in HDFS:

>>source ~/mrjob-env/bin/activate
>>hdfs dfs -rm -r -f /user/nidhi/weather_output
>>python ~/weather.py -r hadoop hdfs:///user/nidhi/weather/weather.csv \-o hdfs:///user/nidhi/weather_output

4. View the output in HDFS and copy it to a local text file:

>>hdfs dfs -cat /user/nidhi/weather_output/part-00000

>>hdfs dfs -get -f /user/nidhi/weather_output/part-00000 ~/weather_output.txt Execution Output:

```
aiml@ubuntu:~$ start-dfs.sh
Starting namenodes on [localhost]
localhost: namenode is running as process 8693.  Stop it first and ensure /tmp/hadoop-aiml-namenode.pid file is empt
fore retry.
Starting datanodes
localhost: datanode is running as process 8826.  Stop it first and ensure /tmp/hadoop-aiml-datanode.pid file is empt
fore retry.
Starting secondary namenodes [ubuntu]
ubuntu: secondarynamenode is running as process 8990.  Stop it first and ensure /tmp/hadoop-aiml-secondarynamenode.p
ile is empty before retry.
aiml@ubuntu:~$ start-yarn.sh
Starting resourcemanager
resourcemanager is running as process 9197.  Stop it first and ensure /tmp/hadoop-aiml-resourcemanager.pid file is e
 before retry.
Starting nodemanagers
localhost: nodemanager is running as process 9329.  Stop it first and ensure /tmp/hadoop-aiml-nodemanager.pid file t
pty before retry.
aiml@ubuntu:~$ jps
9329 NodeManager
8693 NameNode
8826 DataNode
9197 ResourceManager
8990 SecondaryNameNode
17215 Jps
```

Figure 1: Starting HDFS and YARN - jps shows NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager

```
aiml@ubuntu:~$ cat > ~/weather.csv <<'EOF'
date,city,temperature,humidity
2026-09-01,Mysore,29,65
2026-09-01,Bangalore,27,70
2026-09-01,Chennai,34,75
2026-09-02,Mysore,31,60
2026-09-02,Bangalore,26,72
2026-09-02,Chennai,36,78
2026-09-03,Mysore,28,68
2026-09-03,Bangalore,29,66
2026-09-03,Chennai,35,80
EOF
aiml@ubuntu:~$ cat ~/weather.csv
date,city,temperature,humidity
2026-09-01,Mysore,29,65
```

Figure 2: Creating and displaying the input file weather.csv

```
from mrjob.job import MRJob


class MaxTemperature(MRJob):

    def mapper(self, _, line):
        fields = line.split(',')
        if fields[0] == 'date':
            return
        city = fields[1]
        temperature = int(fields[2])
        yield city, temperature

    def reducer(self, city, temperatures):
        yield city, max(temperatures)


if __name__ == '__main__':
    MaxTemperature.run()
EOF
aiml@ubuntu:~$ cat ~/weather.py
```

Figure 3: weather.py MapReduce program

```
2026-09-02,Chennai,36,78
2026-09-03,Mysore,28,68
2026-09-03,Bangalore,29,66
2026-09-03,Chennai,35,80
aiml@ubuntu:~$ source ~/mrjob-env/bin/activate
(mrjob-env) aiml@ubuntu:~$ hdfs dfs -rm -r -f /user/nidhi/weather_output
Deleted /user/nidhi/weather_output
(mrjob-env) aiml@ubuntu:~$ python ~/weather.py -r hadoop hdfs:///user/nidhi/weather/weather.csv -o hdfs:///user/nidhi/we
ather_output
Using configs in /home/aiml/.mrjob.conf
Looking for hadoop binary in /home/aiml/hadoop/bin...
Found hadoop binary: /home/aiml/hadoop/bin/hadoop
Using Hadoop version 3.5.0
Looking for Hadoop streaming jar in /home/aiml/hadoop...
Found Hadoop streaming jar: /home/aiml/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar
Creating temp directory /tmp/weather.aiml.20260925.064056.104619
uploading working dir files to hdfs:///user/aiml/tmp/mrjob/weather.aiml.20260925.064056.104619/files/wd...
Copying other local files to hdfs:///user/aiml/tmp/mrjob/weather.aiml.20260925.064056.104619/files/
Running step 1 of 1...
  packageJobJar: [/tmp/hadoop-unjar1007641242465134135 6/] [] /tmp/streamjob14025143979758572483.jar tmpDir=null
  Connecting to ResourceManager at /0.0.0.0:8032
  Connecting to ResourceManager at /0.0.0.0:8032
  Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/aiml/.staging/job_1790315640419_0004
  Total input files to process : 1
  number of splits:2
  Submitting tokens for job: job_1790315640419_0004
  Executing with tokens: []
  resource-types.xml not found
  Unable to find 'resource-types.xml'.
  Submitted application application_1790315640419_0004
  The url to track the job: http://ubuntu:8088/proxy/application_1790315640419_0004/
  Running job: job_1790315640419_0004
  Job job_1790315640419_0004 running in uber mode : false
```

Figure 4: Running the MapReduce job on Hadoop with the output directory set to HDFS

```
                Peak Map Virtual memory (bytes)=2751119360
                Peak Reduce Physical memory (bytes)=203431936
                Peak Reduce Virtual memory (bytes)=2754256896
                Physical memory (bytes) snapshot=746176512
                Reduce input groups=3
                Reduce input records=9
                Reduce output records=3
                Reduce shuffle bytes=150
                Shuffled Maps =2
                Spilled Records=18
                Total committed heap usage (bytes)=417333248
                Virtual memory (bytes) snapshot=8256352256
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
job output is in hdfs:///user/nidhi/weather_output
Removing HDFS temp directory hdfs:///user/aiml/tmp/mrjob/weather.aiml.20260925.064056.104619...
Removing temp directory /tmp/weather.aiml.20260925.064056.104619...
(mrjob-env) aiml@ubuntu:~$ hdfs dfs -ls /user/nidhi/weather_output
Found 2 items
-rw-r--r--   1 aiml supergroup          0 2026-09-25 06:44 /user/nidhi/weather_output/_SUCCESS
-rw-r--r--   1 aiml supergroup         40 2026-09-25 06:44 /user/nidhi/weather_output/part-00000
(mrjob-env) aiml@ubuntu:~$ hdfs dfs -cat /user/nidhi/weather_output/part-00000
"Bangalore"     29
"Chennai"       36
"Mysore"        31
(mrjob-env) aiml@ubuntu:~$ hdfs dfs -get -f /user/nidhi/weather_output/part-00000 ~/weather_output.txt
(mrjob-env) aiml@ubuntu:~$
```

Figure 5: Job counters, output files (_SUCCESS and part-00000) in HDFS and the final result

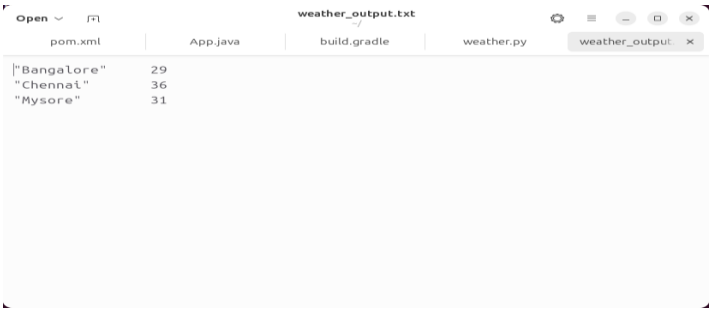


Figure 6: Output copied from HDFS to the local file weather_output.txt and opened in the text editor

Brief Discussion:

The program works in two stages. In the Map stage, every weather record is read and the city and temperature are taken out. Hadoop then shuffles and sorts these pairs so that all temperatures of the same city are grouped together. In the Reduce stage, the maximum temperature of each city is selected.

City	Temperatures (Map output)	Maximum (Reduce output)
Bangalore	27, 26, 29	29
Chennai	34, 36, 35	36
Mysore	29, 31, 28	31

The job counters confirm this flow: Reduce input records = 9 (nine readings), Reduce input groups = 3 (three cities) and Reduce output records = 3 (one maximum per city). Hadoop ran 2 map tasks in parallel and 1 reduce task. The output was saved in HDFS at /user/nidhi/weather_output, which contains the _SUCCESS marker file and the result file part-00000.

Result:

The Hadoop MapReduce job ran successfully and gave the following result:

City	Maximum Temperature (°C)
Bangalore	29
Chennai	36
Mysore	31

Chennai recorded the highest temperature (36°C) among the three cities.

Conclusion:

The Hadoop MapReduce program was successfully implemented using Python and mrjob on a single-node Hadoop cluster running on Ubuntu. The weather data was stored in HDFS and processed with MapReduce on YARN. The program correctly found the maximum temperature for each city, and the output was saved in HDFS and copied to a local text file.